SISTEMA NATYS

Informe de Pruebas de Software

Módulo: Pedido

Integrante	Elias Arma Moises Pereira Edgmairys Aponte Ismael Torres Santiago Loyo
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	17-10-2026

1. Introducción

El presente informe documenta el proceso integral de aseguramiento de calidad (QA) y pruebas de software aplicadas a cada uno de los módulos que integran el **Sistema Naty's**. El objetivo principal es verificar y validar el correcto funcionamiento de los flujos de trabajo del sistema, incluyendo la gestión completa del ciclo de vida de los datos (registro, consulta, modificación, eliminación, aprobación y cancelación de acciones).

Para garantizar un nivel óptimo de estabilidad, rendimiento y seguridad, se implementó una estrategia de evaluación multinivel que abarca las siguientes tipologías de pruebas:

- **Pruebas Unitarias y de Humo:** Ejecutadas directamente sobre la capa de modelo mediante **PHPUnit** para verificar el comportamiento aislado de las funciones core y asegurar la estabilidad básica antes de fases avanzadas.
- **Pruebas de Integración y Aceptación:** Realizadas con **PHPUnit** y **Codeception**, validando la interacción entre componentes y garantizando que los módulos cumplan con los criterios de aceptación y requerimientos del usuario.
- **Pruebas de Backend y Seguridad:** Implementadas con **Postman** para auditar el comportamiento de las APIs, el manejo de respuestas HTTP y la resistencia ante vulnerabilidades en los endpoints.
- **Pruebas de Interfaz de Usuario (UI):** Automatizadas con **Selenium** para simular la navegación e interacción directa del usuario en el navegador.
- **Pruebas de Rendimiento y Carga:** Llevadas a cabo mediante **Apache JMeter**, evaluando los tiempos de respuesta y la estabilidad del sistema bajo volúmenes de concurrencia y carga de datos.

Modulo: Pedido

El módulo Pedido se verificó con cinco pruebas unitarias (UT-01 a UT-05) que cubren la validación de acciones y el CRUD completo mediante mocks, de modo que se ejecutan sin servidor ni base de datos. Finalmente, una prueba de integración (IT-01) comprobó que el modelo consulta y devuelve pedidos reales desde la base de datos MySQL.

1.1 Resumen de resultados

ID	Caso de prueba	Acción	Estado
UT-01	Acción inválida retorna error de estructura	manejarAccion	**Aprobado**
UT-02	Registrar pedido con datos inválidos es rechazado por el validador	registrar	**Aprobado**
UT-03	Obtener pedido exitosamente mediante mock	obtener	**Aprobado**
UT-04	Modificar pedido exitosamente mediante mock	modificar	**Aprobado**

ID	Caso de prueba	Acción	Estado
UT-05	Eliminar pedido exitosamente mediante mock	eliminar	**Aprobado**
Suite	Archivo completo tests/unitarias/PedidoTest.php (5 pruebas unitarias)	todas	**5 de 5 Aprobado**
IT-01	Integración: consultar pedidos en la base de datos real	consultar	**Aprobado**
API-01	Backend: obtener un pedido por su ID (Postman)	obtener	**Aprobado**
SM-01	Humo: el módulo de Pedido inicializa y carga correctamente	todas	**Aprobado**
UI-01	Interfaz y sistema: el formulario Modificar muestra los datos del pedido (Selenium IDE)	modificar	**Aprobado**
SEC-01	Seguridad: obtener un pedido sin sesión (Postman)	obtener	**Aprobado**
SEC-02	Seguridad: consultar el módulo de pedidos sin sesión (Postman)	consultar	**Aprobado**
SEC-03	Seguridad: inyección SQL en el identificador del pedido (Postman)	obtener	**Aprobado**
RD-01	Rendimiento: 50 usuarios consultan el listado de pedidos simultáneamente (JMeter)	login / consultar	**Aprobado**
AC-01	Aceptación: consultar los pedidos y el detalle de uno (Codeception)	consultar / obtener	**Aprobado**

1.2 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Validación de acciones	UT-01 · Acción inválida	**Rechaza comandos no reconocidos devolviendo status = false y el mensaje «Accion Invalida.».**
Crear / Registrar	UT-02 · Registrar pedido con datos inválidos	**El validador detiene el registro cuando faltan datos obligatorios y devuelve status = false.**
Leer	UT-03 · Obtener pedido mediante mock	**Recupera el pedido y sus datos (cliente, monto y estado) desde la consulta simulada.**
Actualizar	UT-04 · Modificar pedido mediante mock	**Actualiza el pedido, restaura el stock anterior y confirma la transacción.**
Eliminar	UT-05 · Eliminar pedido mediante mock	Desactiva el pedido (status = 0), restaura el stock y confirma la transacción.

1.3 Prueba unitaria UT-01: Acción inválida retorna error de estructura

Campo	Detalle
ID	UT-01
Nombre	Acción inválida retorna error de estructura
Tipo de prueba	Unitaria
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/unitarias/PedidoTest.php · test_accion_invalida_retorna_error
Descripción	Verifica que el método centralizador manejarAccion rechace cualquier comando o acción no reconocida ('accion_falsa'), devolviendo un arreglo con status = false y el mensaje «Accion Invalida.» sin afectar la base de datos.
Precondiciones	PHPUnit instalado en el proyecto · Configuración phpunit.xml presente · Entorno aislado sin requerir servidor MySQL activo.
Datos de entrada	**Acción: accion_falsa / JSON: {}**
Pasos	1. Se abrió la terminal CMD en la ruta del proyecto C:\xampp\htdocs\larence. 2. Se verificó PHPUnit con: vendor\bin\phpunit --version 3. Se ejecutó la prueba unitaria específica: vendor\bin\phpunit tests\unitarias\ PedidoTest.php --filter test_accion_invalida_retorna_error
Resultado esperado	El modelo procesa la acción desconocida, devuelve un arreglo con status = false y el mensaje de error «Accion Invalida.».
Resultado obtenido	El modelo devolvió correctamente status = false y el mensaje esperado. PHPUnit validó satisfactoriamente todas las aserciones de la estructura de error.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

1.3.1 Evidencias

Captura 1 – Código del método aislado

```php
/** @test */
public function test_accion_invalida_retorna_error(): void {
    $pedido = new Pedido();
    $resultado = $pedido->manejarAccion('accion_falsa', '{}');

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
    $this->assertEquals('Accion Invalida.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_accion_invalida_retorna_error
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.024, Memory: 8.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 5.
```

Prueba aprobada con XAMPP apagado

2.1 Prueba unitaria UT-02: Registrar pedido con datos inválidos

ID	UT-02
Nombre	Registrar pedido con datos inválidos
Tipo de prueba	Unitaria
Acción evaluada	registrar
Archivo y método	tests/unitarias/PedidoTest.php · test_registrar_pedido_con_datos_invalidos
Descripción	Verifica que el método registrar rechace una solicitud con datos incompletos (cliente_id vacío), deteniendo el proceso antes de insertar el pedido y devolviendo status = false.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PHPUnit configurado y operativo en el proyecto.
Simulación (mock)	No requiere simulación de PDO: el validador de datos detiene la operación antes de cualquier acceso a la base de datos.
Datos de entrada	**`Acción: registrar / JSON: {"cliente_id": ""}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/PedidoTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_registrar_pedido_con_datos_invalidos --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El validador detecta la ausencia del dato obligatorio, no registra el pedido y devuelve status = false.
Resultado obtenido	✔️ Registrar pedido con datos invalidos. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo rechazó la solicitud incompleta sin acceder a la base de datos.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_registrar_pedido_con_datos_invalidos(): void {
    $pedido = new Pedido();
    // Simulamos un JSON incompleto para verificar que el validador detenga el registro
    $resultado = $pedido->manejarAccion('registrar', json_encode(['cliente_id' => '']));

    $this->assertIsArray($resultado);
    $this->assertFalse($resultado['status']);
}
```

Captura 2 – Ejecución de UT-02

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_registrar_pedido_con_datos_invalidos
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.032, Memory: 8.00 MB

OK, but there were issues!
Tests: 1, Assertions: 2, PHPUnit Deprecations: 5.
```

Prueba aprobada con XAMPP apagado

2.2 Prueba unitaria UT-03: Obtener pedido exitosamente con mock

ID	UT-03
Nombre	Obtener pedido exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	obtener
Archivo y método	tests/unitarias/PedidoTest.php · test_obtener_pedido_exitosamente_con_mock
Descripción	Verifica que el método obtener recupere un pedido existente mediante una consulta PDO simulada, retornando status = true, el mensaje «Pedido encontrado con éxito.» y los datos del pedido (id_pedido, cliente, monto y estado).
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PedidoTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y la sentencia PDOStatement para interceptar prepare, execute y fetch, devolviendo un pedido con id_pedido = 1, id_cliente = 5, monto_total_pedido = 150.00, nombre_estado = «Pendiente» y status = 1.
Datos de entrada	**`Acción: obtener / JSON: {"id": 1, "cliente_id": 5, "dias_credito": 15, "fecha": "2026-10-20", "estado": 1, "direccion_entrega": "Calle Principal #123", "telefono": "04121234567", "total": 150.00, "productos": [1], "cantidades": [100], "precios": [1.50]}`**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/PedidoTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_obtener_pedido_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo devuelve status = true, el mensaje «Pedido encontrado con éxito.» y los datos del pedido consultado (id_pedido = 1) sin requerir conexión física a la base de datos.
Resultado obtenido	✔️ Obtener pedido exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo devolvió los datos esperados mediante el aislamiento con mocks.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_obtener_pedido_exitosamente_con_mock(): void {
    // 1. Mock de PDOStatement
    $stmtMock = $this->createMock(PDOStatement::class);
    $stmtMock->expects($this->once())
             ->method('execute')
             ->willReturn(true);

    $stmtMock->expects($this->once())
             ->method('rowCount')
             ->willReturn(1);

    $stmtMock->expects($this->once())
             ->method('fetch')
             ->willReturn([
                'id_pedido' => 1,
                'id_cliente' => 5,
                'monto_total_pedido' => 150.00,
                'nombre_estado' => 'Pendiente',
                'status' => 1
            ]);

    // 2. Mock de PDO
    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->expects($this->once())
            ->method('prepare')
            ->willReturn($stmtMock);

    // 3. Mock parcial de Pedido
    $pedidoMock = $this->getMockBuilder(Pedido::class)
                       ->onlyMethods(["getConnectionNegocio", "closeConnection"])
                       ->getMock();

    $pedidoMock->expects($this->any())
               ->method("getConnectionNegocio")
               ->willReturn($pdoMock);

    $pedidoMock->expects($this->any())
               ->method("closeConnection")
               ->willReturn(null);

    // 4. JSON estructurado correctamente para pasar el validador
    $jsonDatos = json_encode([
        'id'               => 1,
        'cliente_id'       => 5,
        'dias_credito'     => 15,
        'fecha'            => date("Y-m-d"),
        'estado'           => 1,
        'direccion_entrega'=> 'Calle Principal □#123',
        'telefono'         => '04121234567',
        'total'            => 150.00,
        'productos'        => [1],
        'cantidades'       => [100],
        'precios'          => [1.50]
    ]);

    $resultado = $pedidoMock->manejarAccion('obtener', $jsonDatos);

    // 5. Aserciones
    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status'], 'Error devuelto por el modelo: ' . ($resultado['msj'] ?? 'Sin mensaje'));
    $this->assertEquals("Pedido encontrado con éxito.", $resultado['msj']);
    $this->assertEquals(1, $resultado["data"]['id_pedido']);
}
```

Captura 2 – Ejecución de UT-03

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_obtener_pedido_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.051, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 6, PHPUnit Deprecations: 5.
```

Prueba aprobada con XAMPP apagado

2.3 Prueba unitaria UT-04: Modificar pedido exitosamente con mock

ID	UT-04
Nombre	Modificar pedido exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/unitarias/PedidoTest.php · test_modificar_pedido_exitosamente_con_mock

Descripción	Verifica que el método modificar actualice los datos del pedido, restaure el stock del detalle anterior, registre la bitácora y confirme la transacción, devolviendo status = true y el mensaje «Pedido actualizado con éxito.» sin requerir base de datos física.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PedidoTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO y las sentencias PDOStatement para interceptar la consulta del pedido, la actualización (UPDATE pedidos), la lectura del detalle anterior, la verificación de stock (500 unidades) y la confirmación de la transacción (beginTransaction/commit).
Datos de entrada	**Acción: modificar / JSON: {"id": 1, "cliente_id": 5, "dias_credito": 15, "fecha": "2026-10-20", "estado": 1, "direccion_entrega": "Calle Principal #123", "telefono": "04121234567", "total": 160.00, "productos": [1], "cantidades": [100], "precios": [1.60]}**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/PedidoTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_modificar_pedido_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo procesa la actualización, restaura el stock previo, confirma la transacción y devuelve status = true con el mensaje «Pedido actualizado con éxito.».
Resultado obtenido	✔️ Modificar pedido exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo actualizó el pedido y confirmó la transacción simulada.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_modificar_pedido_exitosamente_con_mock(): void {
    // Mock SELECT del pedido para la bitácora
    $stmtSelectBitacora = $this->createMock(PDOStatement::class);
    $stmtSelectBitacora->method('execute')->willReturn(true);
    $stmtSelectBitacora->method('rowCount')->willReturn(1);
    $stmtSelectBitacora->method('fetch')->willReturn([
        'id_pedido' => 1,
        'id_cliente' => 5,
        'monto_total_pedido' => 150.00
    ]);

    // Mock UPDATE principal
    $stmtUpdate = $this->createMock(PDOStatement::class);
    $stmtUpdate->method('execute')->willReturn(true);

    // Mock consulta de detalle anterior para restaurar stock
    $stmtDetalleAnterior = $this->createMock(PDOStatement::class);
    $stmtDetalleAnterior->method('execute')->willReturn(true);
    $stmtDetalleAnterior->method('fetchAll')->willReturn([
        ['id_producto' => 1, 'cantidad' => 100]
    ]);

    // Mock consulta stock del producto
    $stmtStock = $this->createMock(PDOStatement::class);
    $stmtStock->method('execute')->willReturn(true);
    $stmtStock->method('fetchColumn')->willReturn(500);
    $stmtStock->method('fetch')->willReturn(['stock' => 500, 'nombre_producto' => 'Producto Test']);

    // Mock PDO con callback
    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelectBitacora, $stmtUpdate, $stmtDetalleAnterior, $stmtStock) {
        if (strpos($sql, 'UPDATE pedidos') !== false) return $stmtUpdate;
        if (strpos($sql, 'SELECT * FROM detalle_pedidos') !== false) return $stmtDetalleAnterior;
        if (strpos($sql, 'SELECT stock') !== false || strpos($sql, 'SELECT id_producto') !== false) return $stmtStock;
        if (strpos($sql, 'UPDATE cuenta_x_cobrar') !== false || strpos($sql, 'DELETE FROM detalle_pedidos') !== false || strpos($sql, 'INSERT INTO detalle_pedidos') !== false || strpos($sql, 'UPDATE productos') !== false) return $stmtUpdate;
        return $stmtSelectBitacora;
    });

    $pdoMock->method('beginTransaction')->willReturn(true);
    $pdoMock->method('commit')->willReturn(true);

    $pedidoMock = $this->getMockBuilder(Pedido::class)
                       ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                       ->getMock();

    $pedidoMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $pedidoMock->method('closeConnection')->willReturn(null);

    // JSON completo con todos los campos validados
    $jsonDatos = json_encode([
        'id'              => 1,
        'cliente_id'      => 5,
        'dias_credito'    => 15,
        'fecha'           => date('Y-m-d'),
        'estado'          => 1,
        'direccion_entrega'=> 'Calle Principal □ #123',
        'telefono'        => '04121234567',
        'total'           => 160.00,
        'productos'       => [1],
        'cantidades'      => [100],
        'precios'         => [1.60]
    ]);

    $resultado = $pedidoMock->manejarAccion('modificar', $jsonDatos);

    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status'], 'Error devuelto por el modelo: ' . ($resultado['msj'] ?? 'Sin mensaje'));
    $this->assertEquals('Pedido actualizado con éxito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-04

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_modificar_pedido_exitosamente_con_mock
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.041, Memory: 10.00 MB

OK, but there were issues!
Tests: 1, Assertions: 3, PHPUnit Deprecations: 5.
```

Prueba aprobada con XAMPP apagado

2.4 Prueba unitaria UT-05: Eliminar pedido exitosamente con mock

ID	UT-05
Nombre	Eliminar pedido exitosamente con mock
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/unitarias/PedidoTest.php · test_eliminar_pedido_exitosamente_con_mock
Descripción	Verifica que al ejecutar la acción eliminar con un pedido válido, el modelo lo desactive (status = 0), restaure el stock de sus productos, actualice la cuenta por cobrar y confirme la transacción, devolviendo status = true y el mensaje «Pedido eliminado con éxito.».
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · PedidoTest.php configurado con mocks de PDO y PDOStatement.
Simulación (mock)	Se simuló la conexión PDO para interceptar la consulta del pedido, la desactivación (UPDATE pedidos SET status = 0), la restauración de stock, la actualización de la cuenta por cobrar y la confirmación de la transacción.

Datos de entrada	**Acción: eliminar / JSON: {"id": 1, "cliente_id": 5, "dias_credito": 15, "fecha": "2026-10-20", "estado": 1, "direccion_entrega": "Calle Principal #123", "telefono": "04121234567", "total": 150.00, "productos": [1], "cantidades": [100], "precios": [1.50]}**
Pasos	1. Se aseguró la inclusión del método de prueba en tests/unitarias/PedidoTest.php. 2. Se detuvieron los servicios de Apache y MySQL en el panel de XAMPP para garantizar el aislamiento. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\unitarias\PedidoTest.php --filter test_eliminar_pedido_exitosamente_con_mock --testdox 4. Se registró la salida obtenida en consola.
Resultado esperado	El modelo desactiva el pedido, restaura el stock, confirma la transacción y devuelve status = true con el mensaje «Pedido eliminado con éxito.».
Resultado obtenido	✔️ Eliminar pedido exitosamente con mock. 1 prueba ejecutada con las aserciones correctas, sin fallos. El modelo confirmó la transacción simulada y desactivó el pedido correctamente.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código del método aislado

```php
/** @test */
public function test_eliminar_pedido_exitosamente_con_mock(): void
    // 1. Stmts simulados para la eliminación
    $stmtSelect = $this->createMock(PDOStatement::class);
    $stmtSelect->method('execute')->willReturn(true);
    $stmtSelect->method('fetch')->willReturn(['id_pedido' => 1, 'id_cliente' => '12345678']);
    $stmtSelect->method('fetchAll')->willReturn([['id_producto' => 1, 'cantidad' => 100]]);
    $stmtSelect->method('fetchColumn')->willReturn(500);

    $stmtAction = $this->createMock(PDOStatement::class);
    $stmtAction->method('execute')->willReturn(true);

    // 2. Mock de PDO
    $pdoMock = $this->createMock(PDO::class);
    $pdoMock->method('prepare')->willReturnCallback(function($sql) use ($stmtSelect, $stmtAction) {
        if (strpos($sql, 'UPDATE pedidos SET status = 0') !== false || strpos($sql, 'UPDATE cuenta_x_cobrar') !== false || strpos($sql, 'UPDATE productos') !== false) {
            return $stmtAction;
        }
        return $stmtSelect;
    });

    $pdoMock->method('beginTransaction')->willReturn(true);
    $pdoMock->method('commit')->willReturn(true);

    // 3. Mock parcial de Pedido
    $pedidoMock = $this->getMockBuilder(Pedido::class)
                       ->onlyMethods(['getConnectionNegocio', 'closeConnection'])
                       ->getMock();

    $pedidoMock->method('getConnectionNegocio')->willReturn($pdoMock);
    $pedidoMock->method('closeConnection')->willReturn(null);

    // 4. Ejecutamos la acción 'eliminar'
    $jsonDatos = json_encode([
        'id' => 1,
        'cliente_id' => '12345678',
        'total' => 150.00,
        'productos' => [1],
        'cantidades' => [100],
        'precios' => [1.50]
    ]);

    $resultado = $pedidoMock->manejarAccion('eliminar', $jsonDatos);

    // 5. Aserciones
    $this->assertIsArray($resultado);
    $this->assertTrue($resultado['status']);
    $this->assertEquals('Pedido eliminado con éxito.', $resultado['msj']);
}
```

Captura 2 – Ejecución de UT-05

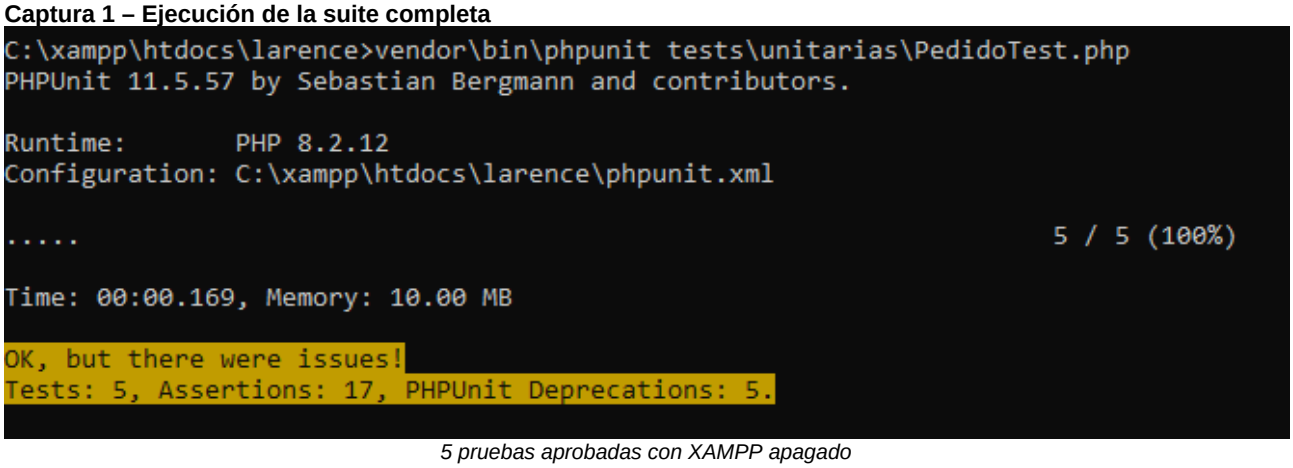


Prueba aprobada con XAMPP apagado

2.5 Ejecución completa de la suite

Comando	vendor\bin\phpunit tests\unitarias\PedidoTest.php --testdox
Precondiciones	XAMPP apagado (Apache y MySQL detenidos).
Resultado esperado	Las 5 pruebas del modelo Pedido aprobadas sin conexión a la base de datos.
Resultado obtenido	5 de 5 pruebas aprobadas, 17 aserciones, sin fallos (1,516 s; 10 MB). PHPUnit mostró 5 avisos «PHPUnit Deprecations» por el uso de la anotación /** @test */ en comentarios, un estilo que PHPUnit 11 marca como obsoleto. No afectan los resultados.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Ejecución de la suite completa



5 pruebas aprobadas con XAMPP apagado

3.1 Prueba de integración IT-01

A diferencia de las pruebas unitarias, la prueba de integración usa el modelo Pedido real conectado a la base de datos MySQL, para comprobar que el código y la base de datos trabajan correctamente en conjunto y que el listado de pedidos coincide con los registros persistidos.

ID	IT-01
Nombre	Integración: consultar pedidos en la base de datos real
Tipo de prueba	Integración (Modelo Pedido + MySQL)
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/integracion/PedidoIntegracionTest.php · test_consultar_pedidos_en_la_base_real
Descripción	Instancia el modelo Pedido real, ejecuta la acción consultar y compara el primer pedido devuelto con los registros de la tabla pedidos en MySQL, comprobando que la respuesta del modelo coincide con la base de datos.

Precondiciones	MySQL activo en XAMPP · Base de datos larence con al menos un pedido registrado.
Datos de entrada	**`Acción: consultar / JSON: {}`**
Pasos	1. Se encendió y verificó el servicio MySQL en el panel de control de XAMPP. 2. Se creó y configuró el archivo tests/integracion/PedidoIntegracionTest.php. 3. Se ejecutó en la terminal: vendor\bin\phpunit tests\integracion\ PedidoIntegracionTest.php --testdox 4. Se registró la salida exitosa obtenida en consola.
Resultado esperado	El modelo devuelve status = true con el listado de pedidos y el primer pedido coincide con un registro real de la tabla pedidos de la base de datos.
Resultado obtenido	1 / 1 prueba ejecutada con éxito. OK (1 test, 3 aserciones). El modelo respondió correctamente y la consulta directa a MySQL confirmó que el pedido devuelto existe en la base de datos real.
Estado	**Aprobado**
Fecha de ejecución	20-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de la prueba de integración

```php
public function test_consultar_pedidos_en_la_base_real(): void {
    // 1. Actuar: Instanciamos el modelo real y ejecutamos la acción 'consultar'
    $pedidoModelo = new Pedido();
    $resultado = $pedidoModelo->manejarAccion('consultar', '{}');

    // 2. Verificar: Comprobamos la respuesta del modelo y la consistencia con la base real
    $this->assertIsArray($resultado);

    if ($resultado['status']) {
        $this->assertNotEmpty($resultado['data'], 'El listado de pedidos devuelto está vacío.');

        // Verificamos que el primer pedido exista físicamente en la BD real
        $idPedido = (int)$resultado['data'][0]['id_pedido'];
        $pedidoDb = $this->db->query("SELECT id_pedido FROM pedidos WHERE id_pedido = $idPedido")->fetch(PDO::FETCH_ASSOC);

        $this->assertNotFalse($pedidoDb, 'El pedido consultado por el modelo no coincide con los registros reales de la base de datos.');
    } else {
        // Si la base no tiene registros activos de pedidos actualmente, se valida el mensaje esperado
        $this->assertEquals('No hay pedidos registrados.', $resultado['msj']);
    }
}
```

Captura 2 – Resultado obtenido de IT-01

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\integracion\PedidoIntegracionTest.php
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.                                                                   1 / 1 (100%)

Time: 00:00.608, Memory: 8.00 MB

OK (1 test, 3 assertions)
```

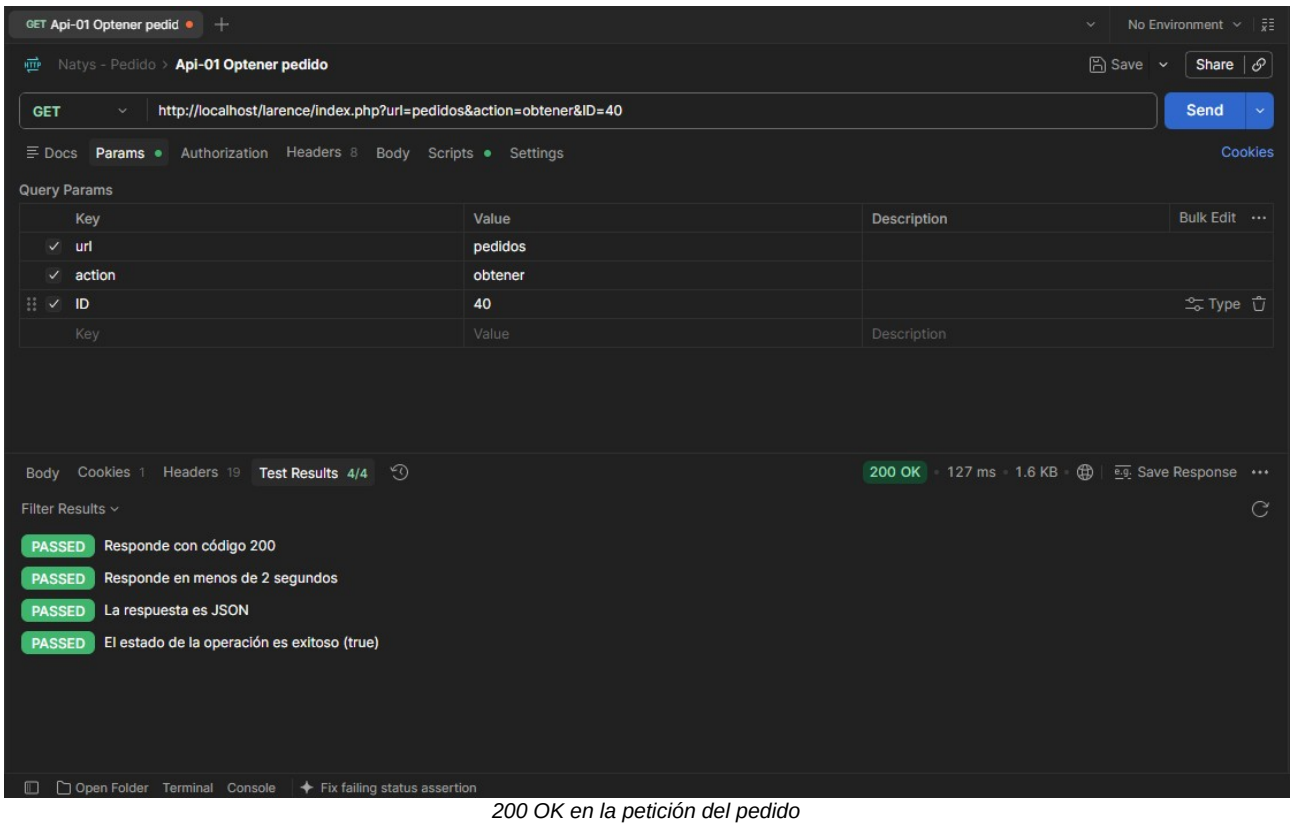
Resultado de la prueba de integración

4.1 Prueba de backend (API) API-01: obtener un pedido por su ID

La prueba de backend se comunica directamente con el controlador mediante peticiones HTTP, sin usar la interfaz web, para verificar que el endpoint del módulo Pedido devuelva el código de estado y el JSON esperados.

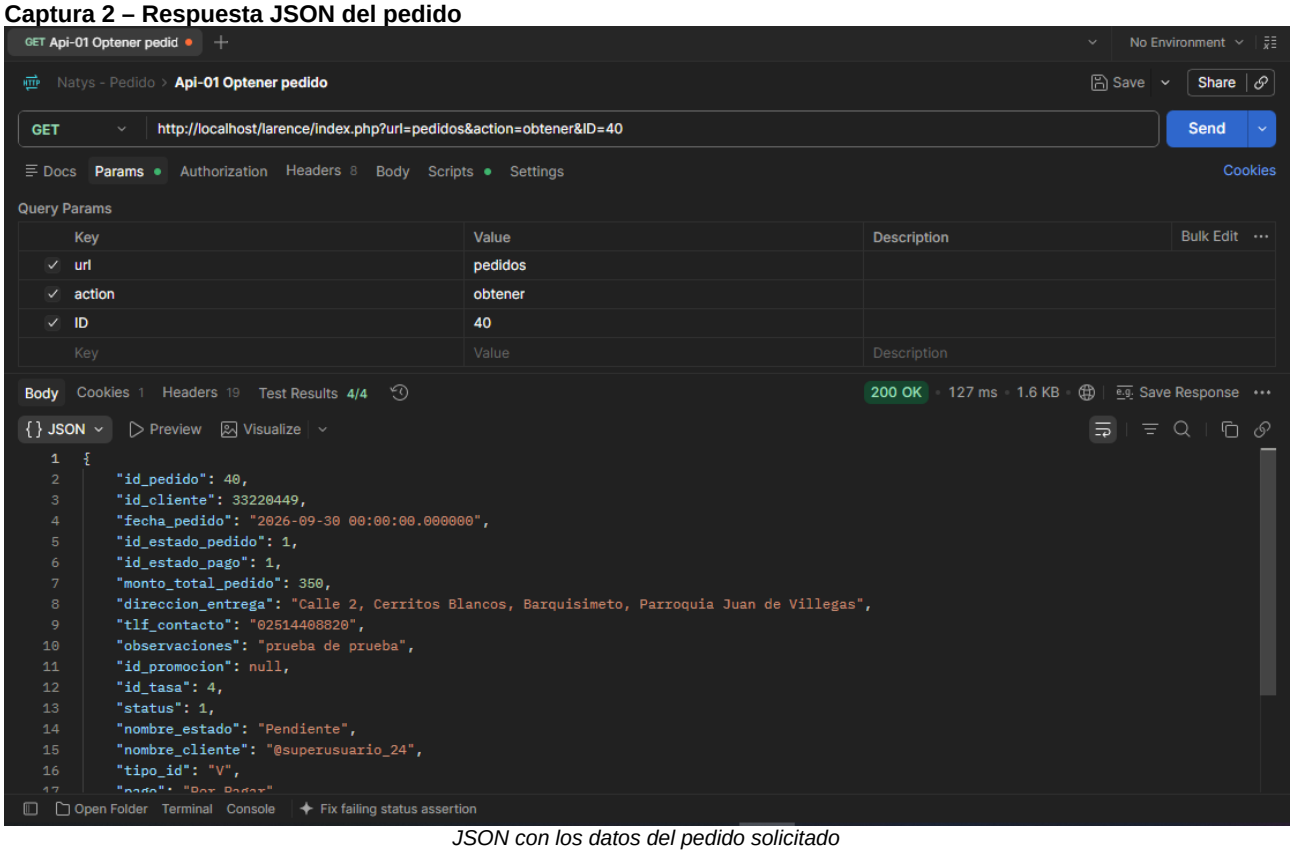
ID	API-01
Nombre	Obtener un pedido por su ID
Tipo de prueba	Backend (API)
Herramienta	Postman · colección «Natys – Pedido»
Petición	`GET http://localhost/larence/index.php?url=pedidos&action=obtener&ID=1`
Descripción	Verifica que el endpoint obtener del PedidoController responda con los datos del pedido solicitado en formato JSON.
Precondiciones	Apache y MySQL activos en XAMPP · sesión iniciada en Natys · cookie de sesión PHPSESSID copiada desde el navegador.
Datos de entrada	`Encabezado: Cookie: PHPSESSID=<sesión activa>` `Parámetro: ID=1`
Pasos	1. Se inició sesión en Natys y se copió la cookie PHPSESSID desde las herramientas del navegador (Application → Cookies). 2. En Postman se creó la colección «Natys – Pedido» y la petición GET «API-01 Obtener pedido». 3. Se agregó el encabezado Cookie y el parámetro ID=1. 4. Se envió la petición (Send) y se verificó el código de estado. 5. Se revisó la respuesta JSON en la pestaña Body (Pretty).
Resultado esperado	Código 200, respuesta JSON en menos de 2 segundos, con los datos del pedido 1: cliente, productos, cantidades, total y estado.
Resultado obtenido	Código 200 OK; respuesta JSON válida con los datos del pedido 1 (cliente, productos, cantidades, total y estado). La consola de Postman muestra una única petición GET con estado 200, sin redirecciones.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – Código de estado de la petición (Postman)



200 OK en la petición del pedido

Captura 2 – Respuesta JSON del pedido



JSON con los datos del pedido solicitado

5.1 Prueba de humo SM-01: Pedido

La prueba de humo es una verificación rápida que se ejecuta antes de las pruebas largas para confirmar que el módulo enciende correctamente: el servidor web, la base de datos, la tabla, el modelo y la página responden sin errores.

ID	SM-01

Nombre	El módulo de Pedido enciende correctamente
Tipo de prueba	Humo (smoke test)
Herramienta	PHPUnit 11.5.57
Archivo	tests/humo/PedidoHumoTest.php (5 verificaciones)
Descripción	Comprueba en menos de un segundo que el servidor web responde, que la base de datos conecta, que existe la tabla pedidos, que el modelo Pedido responde y que la página del módulo carga sin errores de PHP.
Precondiciones	Apache y MySQL encendidos en XAMPP.
Pasos	1. Se encendieron Apache y MySQL en XAMPP. 2. Se creó el archivo tests/humo/PedidoHumoTest.php. 3. Se ejecutó: vendor\bin\phpunit tests/humo/PedidoHumoTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	Las 5 verificaciones aprobadas en pocos milisegundos.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas en menos de un segundo.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

#	Verificación	Si fallara, indicaría…	Resultado
1	El servidor web responde (código 200)	Apache apagado	**Aprobado**
2	La base de datos conecta (SELECT 1)	MySQL apagado o mal configurado	**Aprobado**
3	Existe la tabla pedidos	Base de datos incompleta	**Aprobado**
4	El modelo Pedido responde a consultar	Error grave en el código del modelo	**Aprobado**
5	La página de Pedidos carga sin errores de PHP	Fatal error, Parse error o excepción no capturada	**Aprobado**

Captura 1 – Ejecución de la prueba de humo

```
C:\xampp\htdocs\larence>vendor\bin\phpunit tests\humo\PedidoHumoTest.php --testdox
PHPUnit 11.5.57 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\larence\phpunit.xml

.....                                                               5 / 5 (100%)

Time: 00:00.664, Memory: 8.00 MB

Pedido Humo (Tests\PedidoHumo)
 ✔ Servidor web responde
 ✔ Base de datos conecta
 ✔ Tabla pedidos existe
 ✔ Modelo pedido responde
 ✔ Pagina de pedidos carga sin errores php

OK (5 tests, 9 assertions)
```

5 verificaciones aprobadas, 9 aserciones

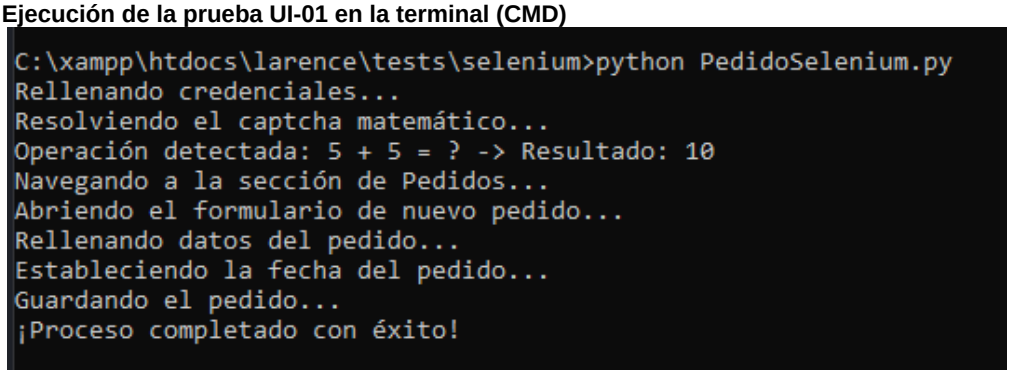
6.1 Prueba de interfaz y sistema UI-01: Pedido

La prueba de interfaz se ejecuta desde el navegador, como lo haría un usuario real: un robot abre la pantalla de Pedido, presiona los botones y verifica lo que aparece. Al recorrer el módulo sin mirar el código, también funciona como prueba de sistema (caja negra).

ID	UI-01
Nombre	El formulario Modificar muestra los datos del pedido
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Herramienta	Selenium IDE (extensión de Firefox)
Descripción	Abre el módulo Pedidos, presiona Modificar en el primer pedido y verifica que el formulario se abra con el cliente, la fecha y el estado cargados.
Precondiciones	Apache y MySQL activos · sesión iniciada en Natys en Firefox · existe al menos un pedido registrado
Pasos	1. Se instaló Selenium IDE en Firefox y se creó el proyecto «Natys Pedidos». 2. Se abrió index.php?url=pedidos con la sesión iniciada. 3. Se presionó Modificar en el primer pedido de la tabla. 4. Se verificaron el título del formulario y los campos cliente, fecha y estado.
Resultado esperado	El robot abre el formulario «Modificar Pedido» con el cliente, los días de crédito, la fecha y el estado del pedido seleccionado.
Resultado obtenido	Los 8 pasos se ejecutaron en verde y la terminal muestra la prueba aprobada.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

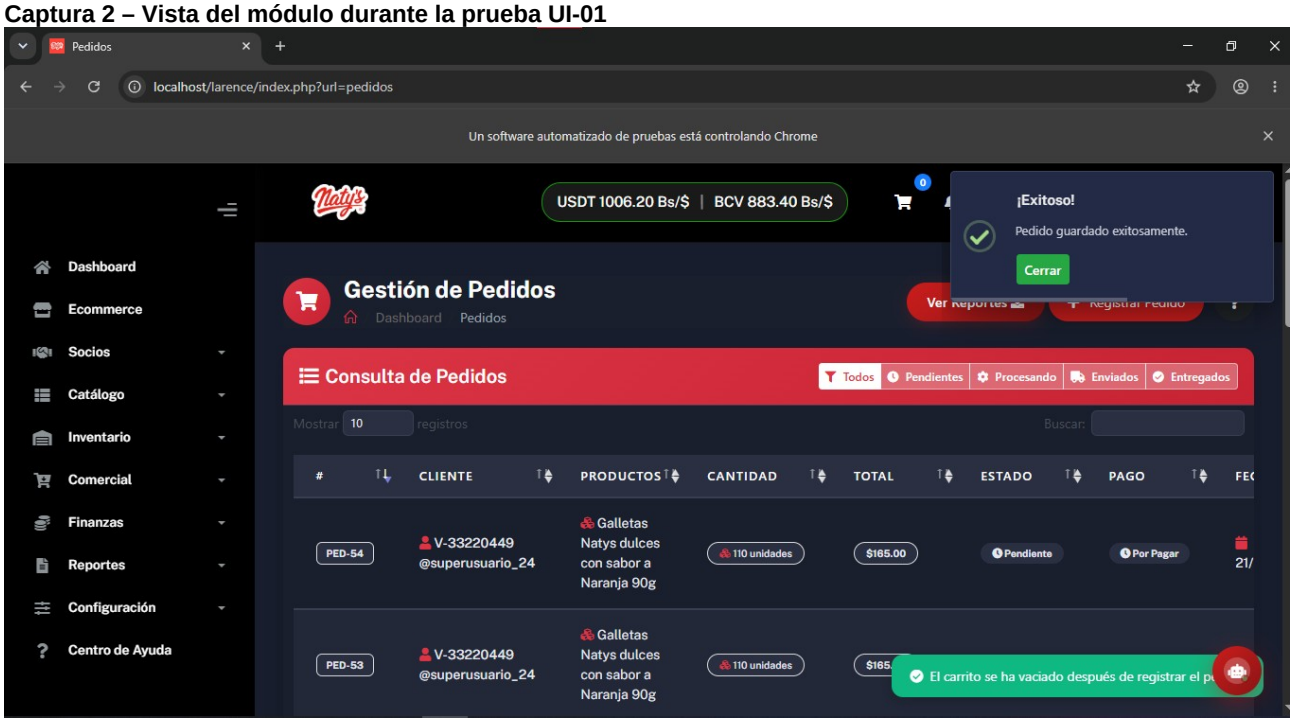
#	Comando	Objetivo	Qué hace
1	open	index.php?url=pedidos	**Abre el módulo Pedidos**
2	set window size	1888x1096	**Fija el tamaño de la ventana**
3	wait for element visible	css=#add-row tbody tr.odd	**Espera a que la tabla muestre los pedidos**
4	click	css=#add-row tbody tr.odd:nth-child(1) .btn:nth-child(2)	**Presiona Modificar en el primer pedido**
5	pause	2000	**Espera la carga de los datos**
6	assert text	id=pedidoModalModificarLabel = «Modificar Pedido»	**Verifica el título del formulario**
7	assert element present	id=clienteIdEdit	**Verifica el campo del cliente**
8	assert element present	id=estadoPedidoEdit	**Verifica el campo del estado**

Captura 1 – Ejecución de la prueba UI-01 en la terminal (CMD)



La terminal muestra la prueba ejecutada y aprobada

Captura 2 – Vista del módulo durante la prueba UI-01



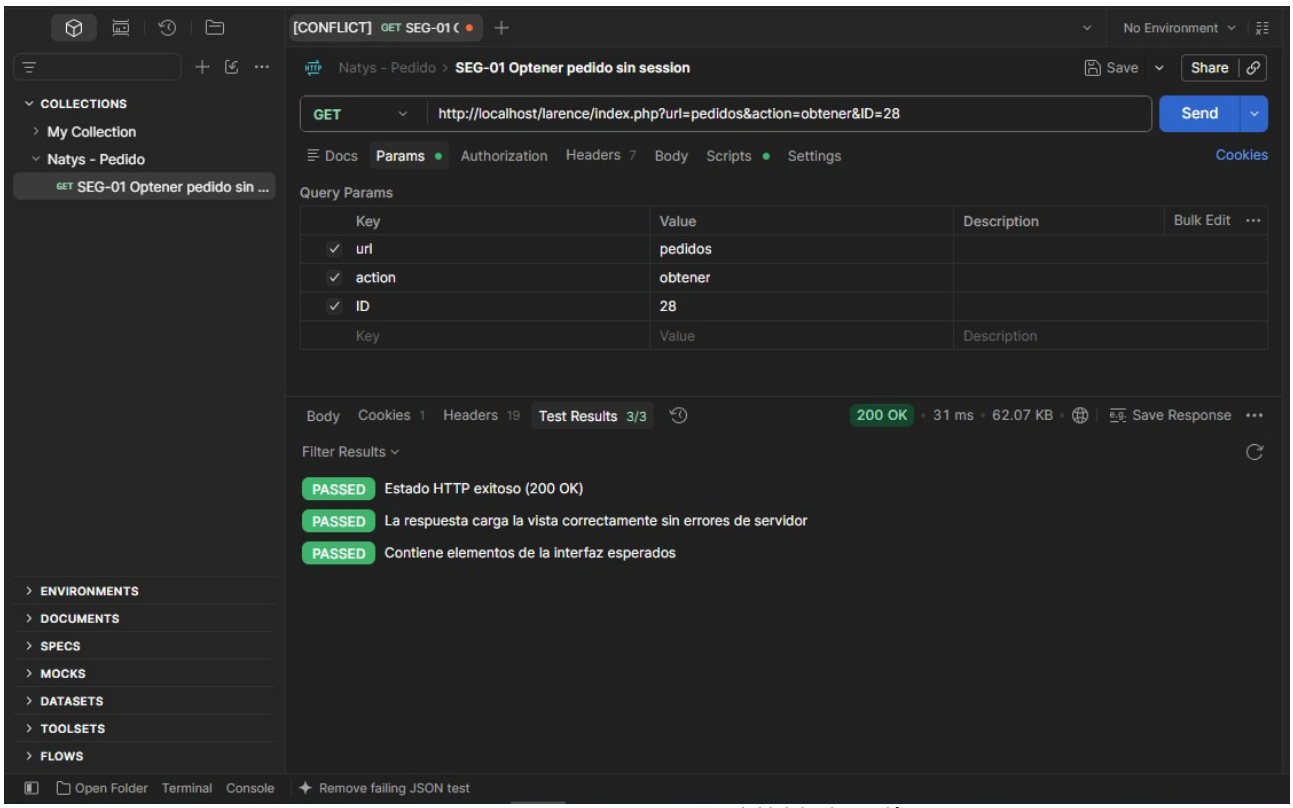
Vista «Gestión de Pedidos» con la tabla de pedidos registrados

7.1 Prueba de seguridad SEC-01: obtener un pedido sin sesión

La prueba intenta obtener un registro del módulo sin haber iniciado sesión, para comprobar que el sistema no expone datos a usuarios anónimos.

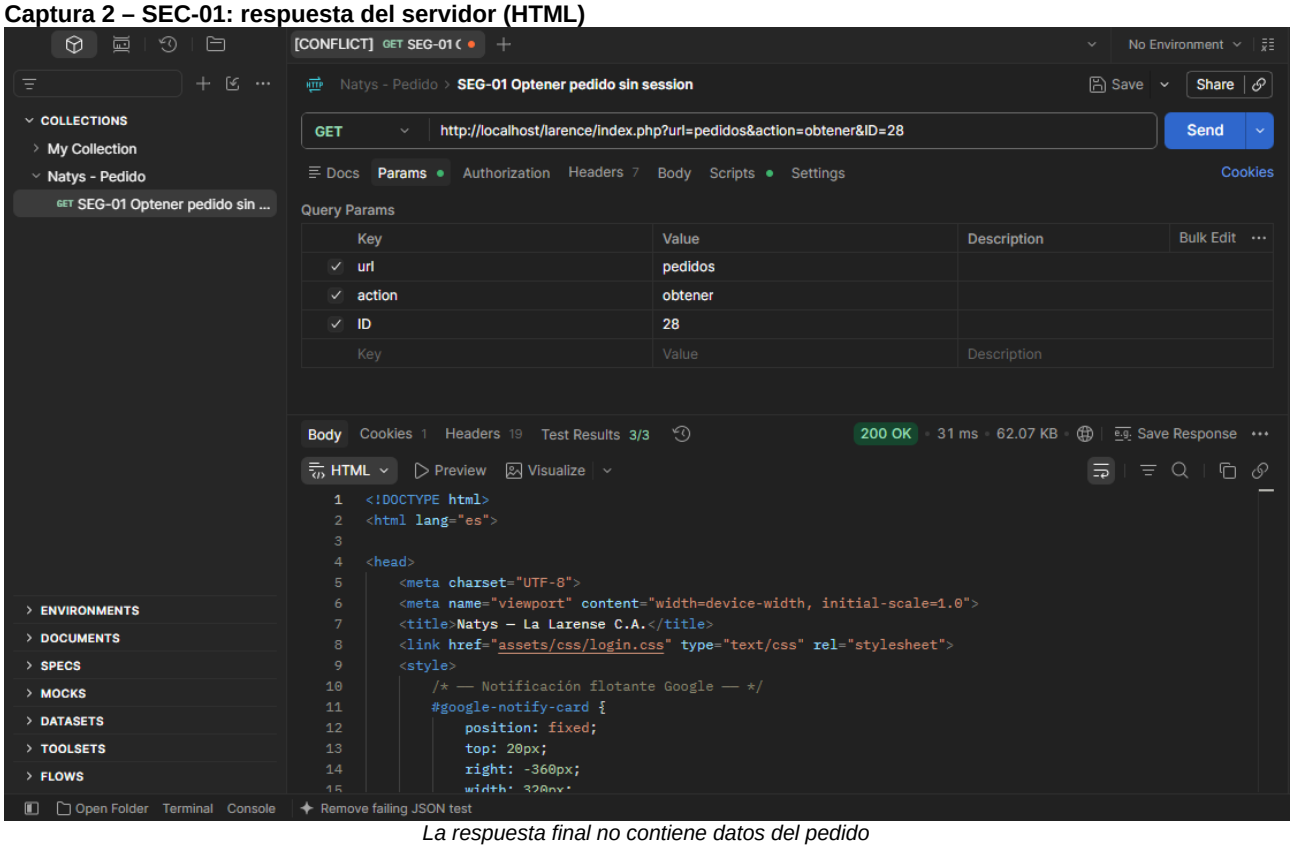
ID	SEC-01
Nombre	Un usuario sin sesión no puede obtener un pedido
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=pedidos&action=obtener&ID=28`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada · redirecciones seguidas por Postman
Pasos	1. Se duplicó la petición API-01 y se renombró como SEC-01. 2. Se eliminó la cookie de autenticación de la petición. 3. Se envió la petición GET y se siguió la redirección. 4. Se registraron el código final y la respuesta HTML del servidor.
Resultado esperado	El servidor redirige al inicio de sesión y la respuesta final no incluye datos del pedido.
Resultado obtenido	200 OK en 31 ms tras seguir la redirección. La respuesta final es la página de inicio de sesión y no contiene datos del pedido. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-01: estatus de la petición



200 OK en 31 ms y respuesta HTML del inicio de sesión

Captura 2 – SEC-01: respuesta del servidor (HTML)



La respuesta final no contiene datos del pedido

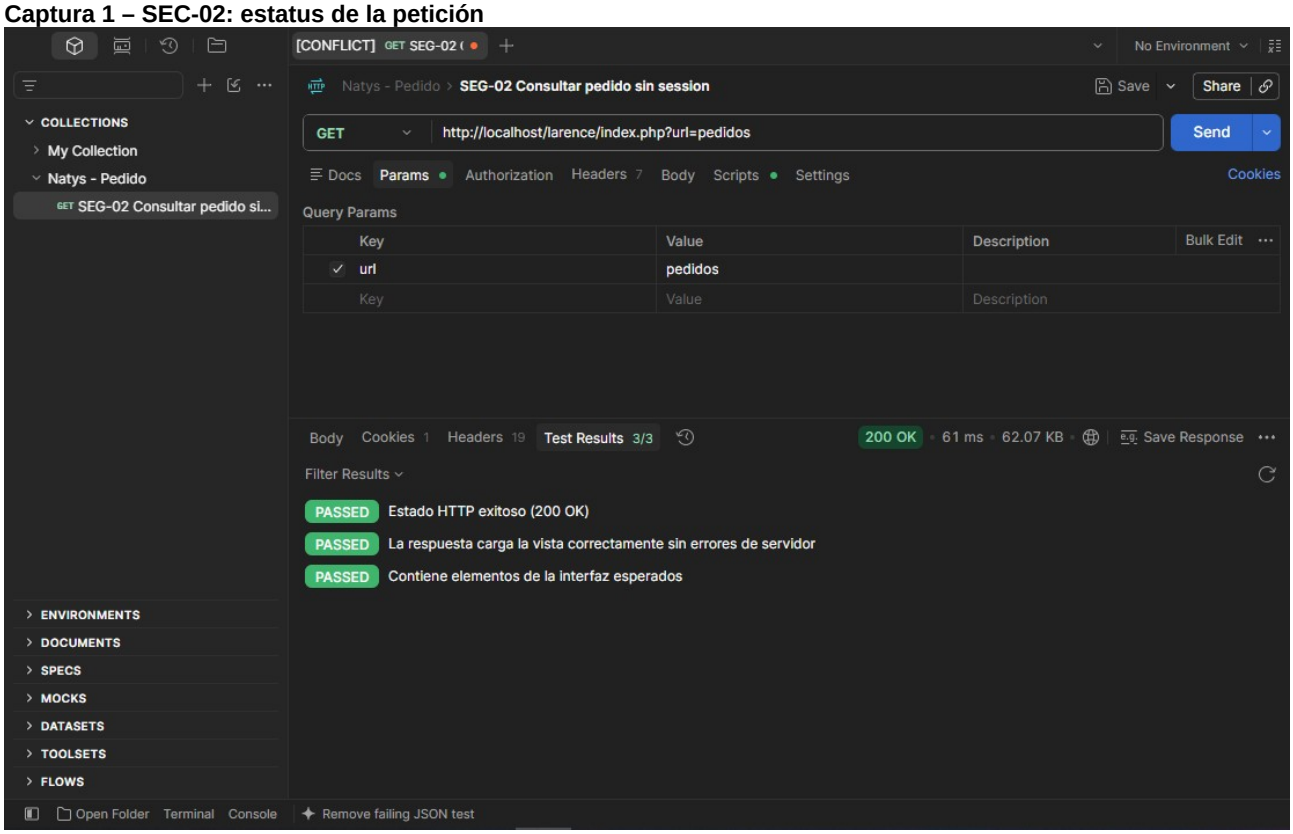
7.2 Prueba de seguridad SEC-02: consultar el módulo de pedidos sin sesión

La prueba intenta consultar el módulo sin haber iniciado sesión, para comprobar que la página y sus datos quedan protegidos detrás del inicio de sesión.

ID	SEC-02

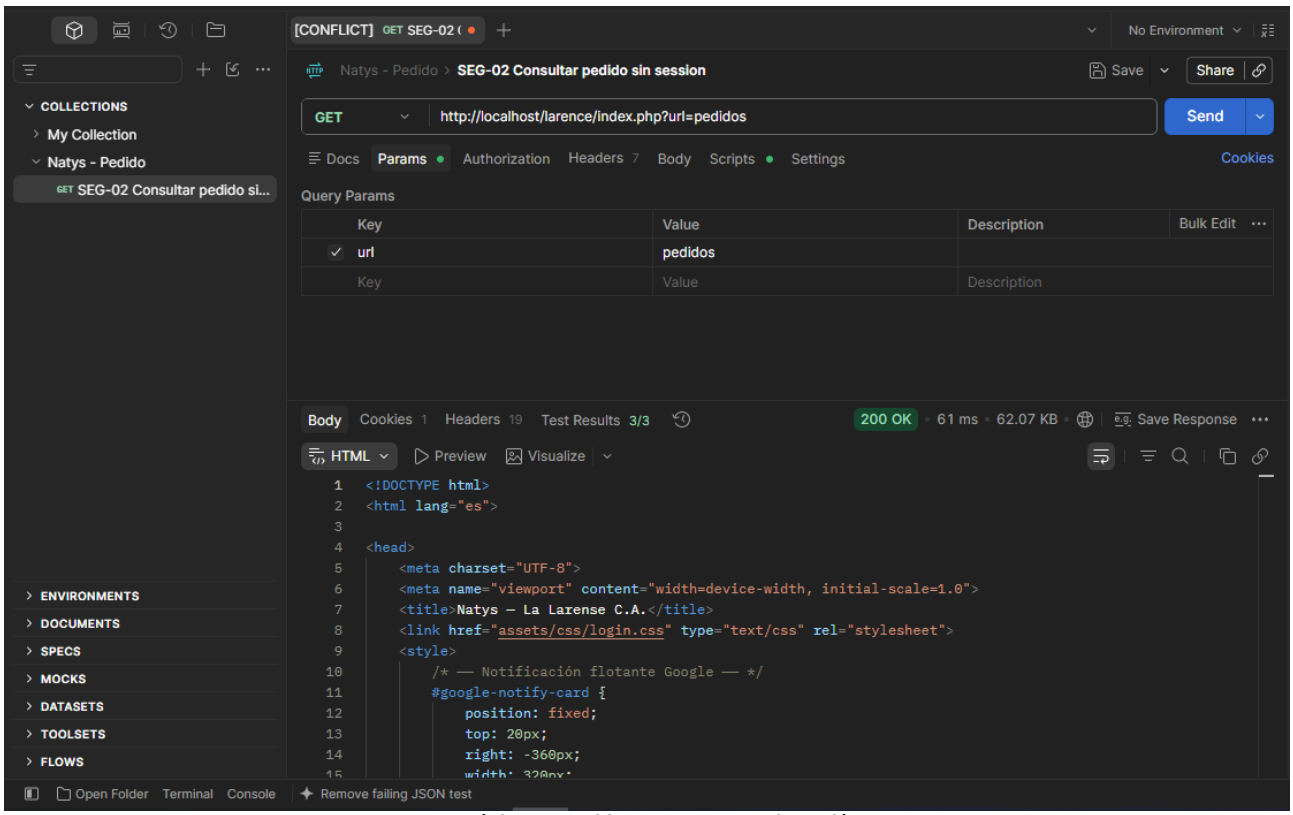
Nombre	Un usuario sin sesión no puede abrir el módulo de pedidos
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=pedidos`
Precondiciones	Apache y MySQL activos · petición sin una sesión autenticada · redirecciones seguidas por Postman
Pasos	1. Se duplicó la petición de consulta del módulo y se renombró como SEC-02. 2. Se eliminó la cookie de autenticación de la petición. 3. Se envió la petición GET y se siguió la redirección. 4. Se registraron el código final y la respuesta HTML del servidor.
Resultado esperado	El servidor redirige al inicio de sesión y la respuesta final no entrega la página del módulo.
Resultado obtenido	200 OK en 61 ms tras seguir la redirección. La respuesta final es la página de inicio de sesión y no entrega la página del módulo. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – SEC-02: estatus de la petición



200 OK en 61 ms y redirección seguida al inicio de sesión

Captura 2 – SEC-02: respuesta del servidor (HTML)



La página protegida no se entrega sin sesión

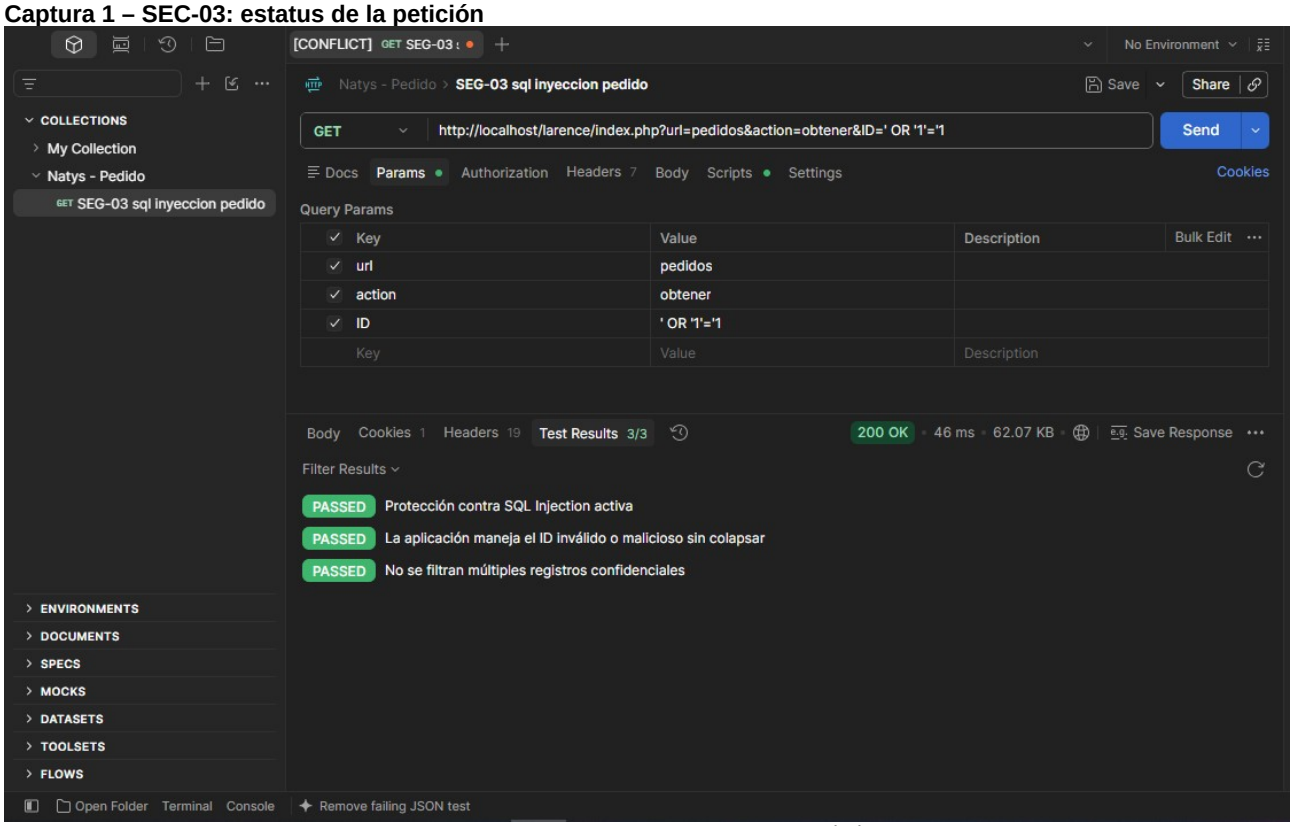
7.3 Prueba de seguridad SEC-03: inyección SQL en el módulo de pedidos

La prueba envía una cadena de inyección SQL en el identificador del registro para comprobar que el sistema la trata como un valor y no como código ejecutable.

ID	SEC-03
Nombre	El identificador del pedido resiste la inyección SQL
Tipo de prueba	Seguridad (inyección SQL)
Herramienta	Postman
Petición	`GET http://localhost/larence/index.php?url=pedidos&action=obtener&ID=1' OR '1'='1`
Descripción	Envía en el parámetro ID una cadena de inyección SQL clásica. Un sistema vulnerable devolvería un listado masivo de pedidos o un error de SQL.
Precondiciones	Apache y MySQL activos · sesión iniciada (cookie de sesión válida en el encabezado).
Datos de entrada	`ID = 1' OR '1'='1`
Pasos	1. Se duplicó la petición de consulta y se renombró como SEC-03. 2. Se reemplazó el valor del parámetro ID por la cadena de inyección. 3. Se envió la petición GET con la sesión iniciada. 4. Se verificó que no apareciera un error SQL ni un listado masivo de registros.
Resultado esperado	Código 200, respuesta HTML, sin mensajes de error SQL y sin un listado masivo de pedidos.
Resultado obtenido	200 OK en 46 ms. La respuesta HTML se procesa sin error SQL y no muestra un listado masivo de pedidos. Las 3 aserciones de Postman se aprobaron.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026

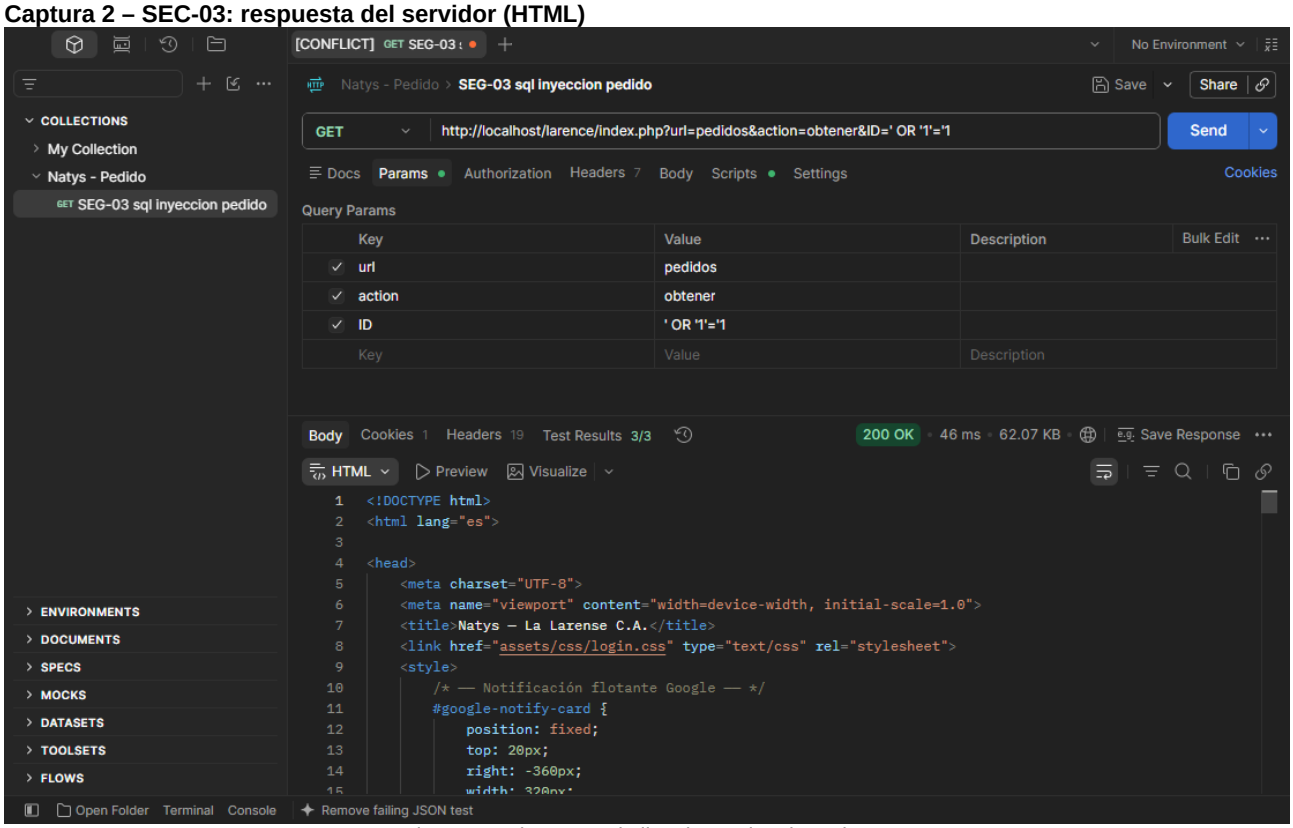
Ejecutado por	Moisés Pereira

Captura 1 – SEC-03: estatus de la petición



200 OK en 46 ms y respuesta HTML controlada

Captura 2 – SEC-03: respuesta del servidor (HTML)



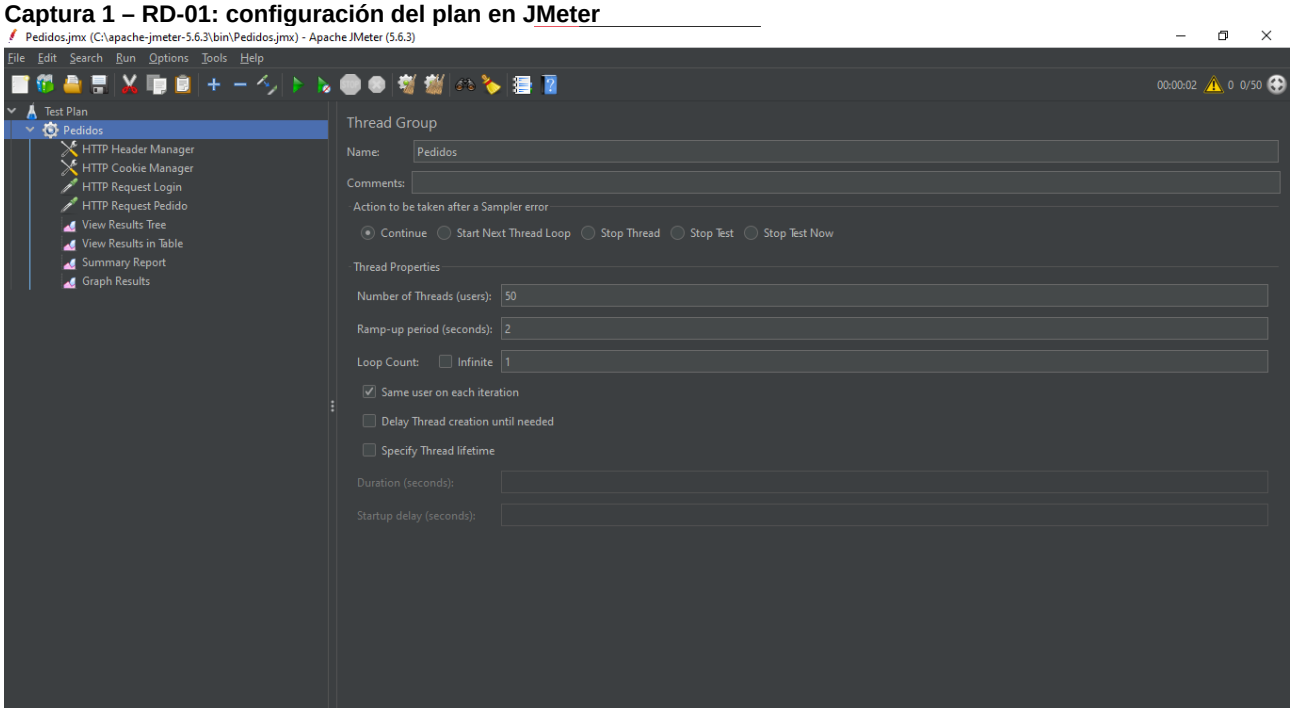
Sin errores de SQL y sin listado masivo de registros

8.1 Prueba de rendimiento RD-01: Pedido

La prueba de rendimiento simula 50 usuarios virtuales que inician sesión y consultan el módulo al mismo tiempo, con el fin de medir el tiempo de respuesta y la estabilidad del sistema bajo carga (RNF 05).

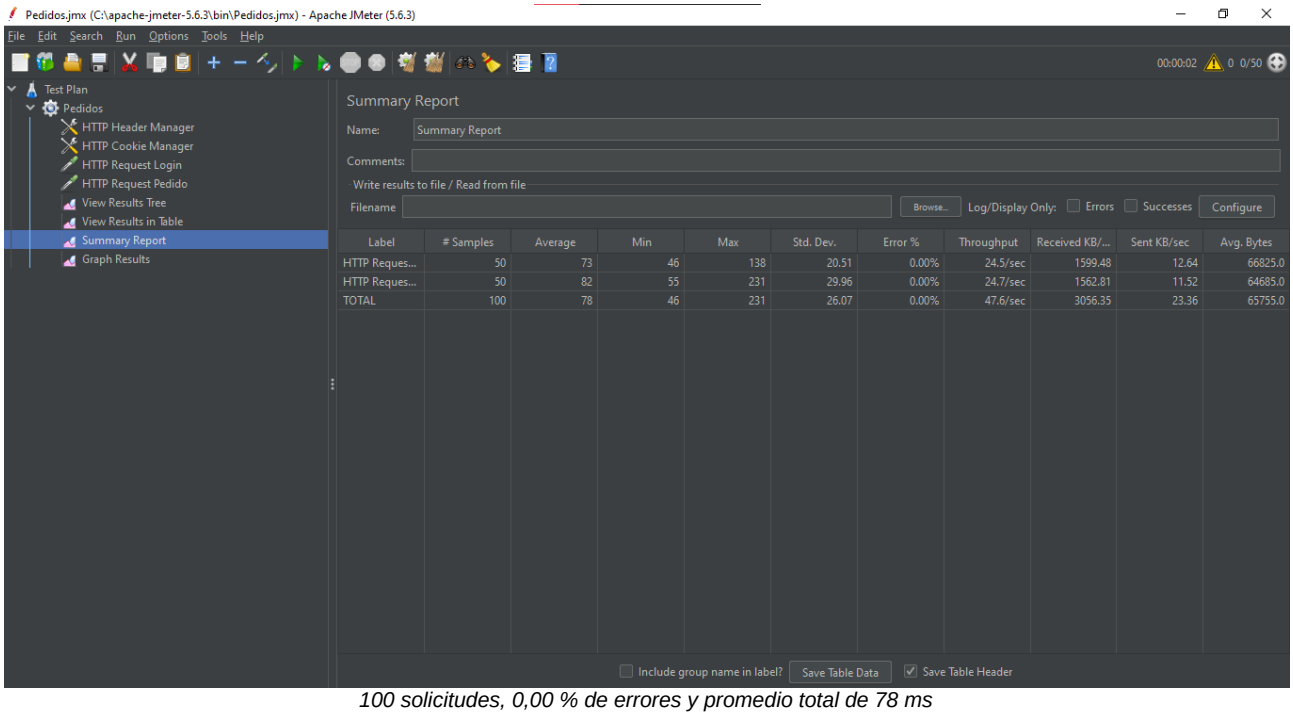
ID	RD-01
Nombre	50 usuarios consultan el listado de pedidos al mismo tiempo
Tipo de prueba	Rendimiento / carga (no funcional)
Herramienta	Apache JMeter 5.6.3 · plan de prueba del módulo Pedido
Petición	`POST http://localhost/larence/index.php?url=autenticator&action=ingresar (login) + GET http://localhost/larence/index.php?url=pedidos`
Configuración	Thread Group: 50 usuarios virtuales · ramp-up de 2 segundos · Loop Count infinito, detenido al completar 100 solicitudes (50 login + 50 módulo).
Precondiciones	Apache y MySQL activos · usuario de prueba válido · HTTP Cookie Manager con la cookie de sesión.
Pasos	1. Se creó el plan de prueba del módulo en Apache JMeter 5.6.3. 2. Se agregaron HTTP Request Defaults (localhost:80, ruta /larence), HTTP Cookie Manager y el Thread Group con 50 usuarios y ramp-up de 2 segundos. 3. Se agregaron las peticiones HTTP Request Login y HTTP Request Pedido. 4. Se agregó el listener Summary Report y se ejecutó el plan con el Thread Group en modo infinito. 5. Se detuvo la ejecución al completar 100 solicitudes y se registraron las métricas.
Resultado esperado	100 solicitudes procesadas, tiempo promedio menor a 2.000 ms y 0 % de errores.
Resultado obtenido	100 solicitudes (50 login + 50 consultas al módulo), promedio de 78 ms (mínimo 46 ms, máximo 231 ms), desviación estándar de 26,07 ms, 0,00 % de errores y 47,6 solicitudes/s.
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – RD-01: configuración del plan en JMeter



Thread Group con 50 usuarios y ramp-up de 2 segundos

Captura 2 – RD-01: Summary Report de JMeter



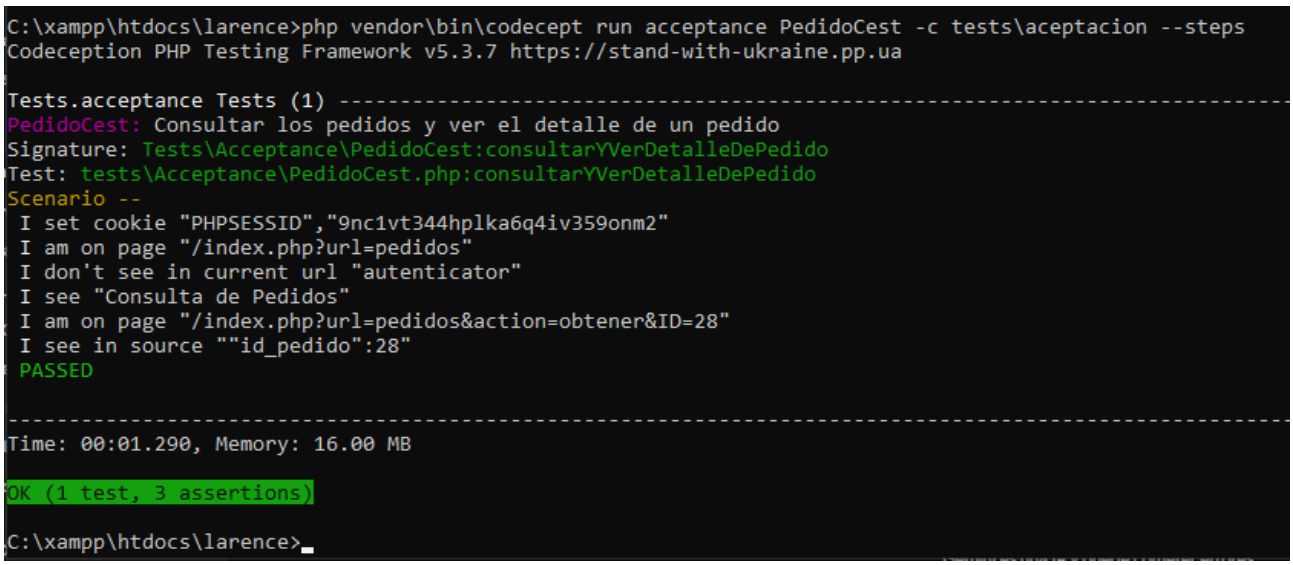
100 solicitudes, 0,00 % de errores y promedio total de 78 ms

9.1 Prueba de aceptación AC-01: Pedido

La prueba de aceptación verifica, desde el navegador, que el módulo cumple el flujo de consulta solicitado por el cliente. Se escribió como una historia de usuario con Codeception y se ejecutó con PhpBrowser.

ID	AC-01
Nombre	El usuario consulta los pedidos y el detalle de uno
Tipo de prueba	Aceptación
Herramienta	Codeception 5.3.7 con el módulo PhpBrowser
Requisitos cubiertos	CRUD Leer: consulta del listado de pedidos y del detalle por ID.
Archivo y método	tests/aceptacion/tests/Acceptance/PedidoCest.php · consultarYVerDetalleDePedido
Precondiciones	Apache y MySQL activos · sesión iniciada en el sistema · cookie PHPSESSID válida.
Pasos	1. Se configuró Codeception 5.3.7 con el módulo PhpBrowser y la URL base del sistema. 2. Se fijó la cookie PHPSESSID de una sesión válida. 3. Se abrió GET /index.php?url=pedidos con amOnPage y se verificó que no redirigiera a autenticator. 4. Se verificó el texto «Consulta de Pedidos», se abrió el detalle del pedido 28 y se comprobó "id_pedido":28 en el código fuente. 5. Se ejecutó: php vendor\bin\codecept run acceptance PedidoCest -c tests\aceptacion --steps.
Resultado esperado	El listado muestra «Consulta de Pedidos» y el detalle del pedido 28 contiene el campo id_pedido con valor 28.
Resultado obtenido	Escenario PASSED: el listado y el detalle del pedido 28 se mostraron correctamente. OK (1 prueba, 3 aserciones) en 1,298 s (16 MB).
Estado	**Aprobado**
Fecha de ejecución	21-10-2026
Ejecutado por	Moisés Pereira

Captura 1 – AC-01: ejecución de la prueba con Codeception



Escenario PASSED: el listado y el detalle del pedido 28 se mostraron correctamente. OK (1 prueba, 3 aserciones) en 1,298 s (16 MB).